

Building Reliable AI Agents with Nemo Agent Toolkit

Design, trace, and evaluate agents that work beyond the demo.

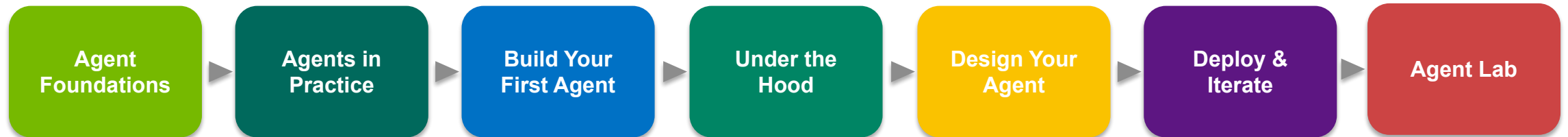
Maryam Najafian, PhD
Brendan Murphy



Course Overview

By the end of this session you will be able to:

- 1 Explain why agents fail and trace the root cause in a span tree
- 2 Compare NAT against LangChain, CrewAI, and AutoGen on concrete capabilities
- 3 Build and run an agent from a single YAML config with real tools
- 4 Choose the right agent type and workflow pattern for a given problem
- 5 Deploy, trace, evaluate, and extend an agent with custom tools
- 6 Benchmark three architectures on GAIA and diagnose differences in Phoenix



Seven sections. Each one builds on the last.

Agent Foundations

Agents in production, core loop, design patterns, and the two paradigms driving the field



Agents in Production

AI agents are moving from pilots into production: Google Cloud reports that 52% of executives say their organizations have deployed AI agents [49] [50].

Code

Coding agents are moving into real engineering workflows. Cognition reported Devin's PR merge rate improved from 34% to 67% [51].

Science

LLM agents designed 92 nanobodies; two showed improved variant binding (Nature) [42].

Finance

Santander and Mastercard completed Europe's first live end-to-end payment executed by an AI agent within a regulated banking framework [52].

Healthcare

Epic reports Penny cut medication prior authorization submission time by 42% at Summit Health, and more than 85% of Epic customers now use Epic AI [53].

Computer Use

GPT-5.4 scored 75.0% on OSWorld-Verified, above the 72.4% human baseline, according to OpenAI [54].

Customer Service

Commerzbank reports its AI assistant resolves more than 70% of customer inquiries independently, with complex cases routed to humans [55].

Robotics

Lightwheel and PeritasAI announced a \$56M program targeting up to 200 humanoid robots in perioperative workflows [56].

Enterprise ROI

Google Cloud reports that 74% of executives saw ROI within the first year of their generative AI deployments [49].

What Makes an Agent

✖ LLM Call

"How many Taylor Swift studio albums as of late 2025?"

→ "11" (wrong: stale answer; missed the 12th album)

- Single pass · No tools · No verification
- Fast, but unreliable for fresh facts, missing training data, or hallucination-prone questions

vs

✓ Agent

Same question

→ Searches the web → verifies count → "12, with source"

- Uses tools · Verifies answer · Cites source
- More accurate, but higher latency and token cost

Tradeoff: tool use improves freshness and verification, but adds latency, tokens, and failure points.

If each step is 95% reliable: 10 steps ≈ 60% success, 20 steps ≈ 36%. Error propagation is the bottleneck [46]

Properties that separate an agent:

Tool Use

Calls APIs, search, databases, or code interpreters

Memory

Carries useful context across steps

Planning

Breaks a goal into executable subtasks

Self-Correction

Checks outputs and retries when needed

The loop is what makes it an agent. Managing the loop is the engineering problem.

How the Agent Loop Works

An agent uses an augmented LLM loop: reason over context, call tools, observe results, and decide or route the next step [10, 46].

Augmented LLM

LLM

Reasons, plans, decides

Tools

Search, code, APIs

Memory

Context across iterations

Retrieval

External knowledge on demand

The Agent Loop

1. Think

Reason over current context.
Use the available context to choose the next action.

2. Act

Emit a structured tool call.
Search, code execution, or API call
through the runtime.

3. Observe

Append tool result to context.
The runtime feeds the observation back for the next step.

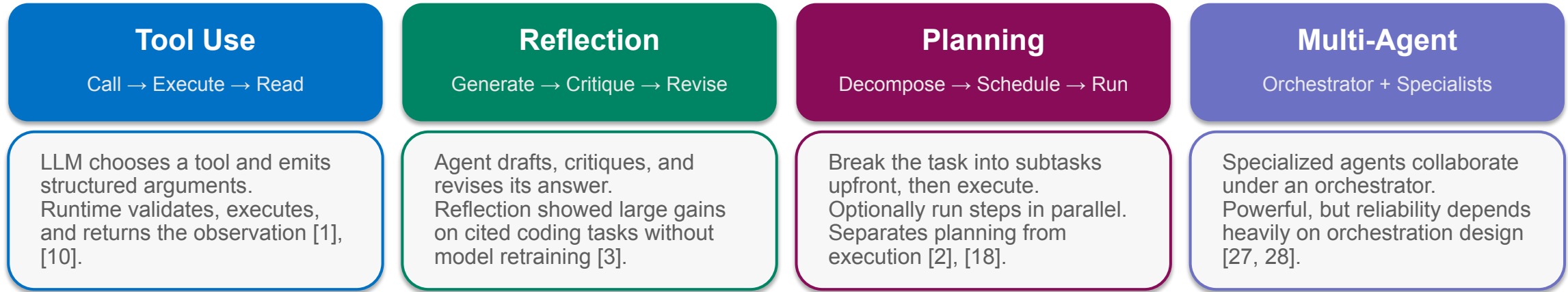
4. Decide (Re-evaluate)

Stop, continue, or route.
If the task is done, return the answer.
Otherwise, continue the loop.

"Find the simplest solution possible. Add complexity only when you have evidence you need it." [10]

More steps can add autonomy, but also add failure points. The engineering challenge is managing the loop around the model.

Four Core Agentic Patterns



**These four compose into richer design patterns. NAT implements several as configurable agent types.
The key insight: agent architecture can move results as much as model choice.**

Agent design can move benchmark results dramatically: one cited HumanEval setup reports GPT-3.5 in an agentic workflow at 95% vs. GPT-4 zero-shot at 67% [18].

Two Agentic Paradigms: Workflows vs Model-Driven

Two designs coexist in production: workflows define the path in code; model-driven agents choose the next step at runtime.

Workflows (Predefined Code Paths) [10]

- Developer designs the control flow; model executes within it
- Tool calls happen through explicit workflow steps
- Predictable, auditable, debuggable
- Frameworks: LangGraph, CrewAI, LlamaIndex Workflows, Google ADK, Microsoft AutoGen, Amazon Bedrock Agents

Model-Driven Agents (Autonomous Agent Loop) [47]

- Model reasons over state, selects tools or code actions, observes results, and loops until completion [10, 47]
- RLVR improved reasoning in DeepSeek-R1-Zero [29]
- Agent-trajectory tuning: 37.2% SWE-Bench Verified with a 32B model [48]
- In production: Cursor, Claude Code, Devin [51], OpenClaw-style autonomous loops [21]

Post-training strengthens model-driven agents by improving reasoning, tool use, and multi-step decisions.

PPO [12] (2017)

PPO-style RLHF: policy optimization with reward/value models. Powerful but costly.

DPO [13] (2023)

Learns directly from preference pairs, without a critic or online RL. Simple and efficient, but less expressive than reward-based RL.

GRPO [14] (2024)

Compares multiple answers from the same prompt and rewards the better ones. No critic model needed, so memory cost is lower.

GSPO [15] (2025)

Optimizes whole output sequences instead of individual tokens. Improves stability for long answers and MoE models. More stable than GRPO.

Where the Field Is Heading

Six shifts shaping agent engineering in 2025-2026, from NeurIPS 2025 and industry research.

Architecture > Model

Major gains increasingly come from the system around the model: orchestration, routing, tool selection, memory, and trajectory design.

Post-training also improves how models plan, use tools, and recover from feedback.

Memory Beyond Context

Agents need more than a bigger context window: they need working memory for the current task, episodic history from past interactions, semantic facts, and procedural skills [20].

MemGPT treats context like managed memory [24], while G-Memory and A-MEM show how agent memories can be structured, retrieved, and evolved over time [33, 34].

Code as Action

Instead of issuing one tool call at a time, agents can write executable code as the action.

CodeAct shows this can improve success by letting agents compose loops, branching, and error handling directly in code [5].

Safety in the Decision Layer

Agent safety has to follow the action path.

Any time an agent can choose tools, pass arguments, call APIs, or act on results, we need guardrails and evaluation across the full trajectory, not just the original prompt [17, 40].

Eval and Self-Healing

Public agent benchmarks are useful, but they are not your production workflow.

Reliable agents need task-specific evals, traces, and feedback loops that can retry, repair, reroute, or fall back when the loop fails [7, 30, 46].

Protocols: MCP + A2A

MCP standardizes how agents connect to tools and context. A2A defines a protocol for agents to communicate across systems [25, 26].

Together they make agents more composable across tools, apps, and teams.

Agents are software systems first: the LLM makes some decisions, but reliability comes from the orchestration, state, tools, evals, and software engineering around it [10, 46].

Agents in Practice

From demo to production: why agents fail and how to fix it

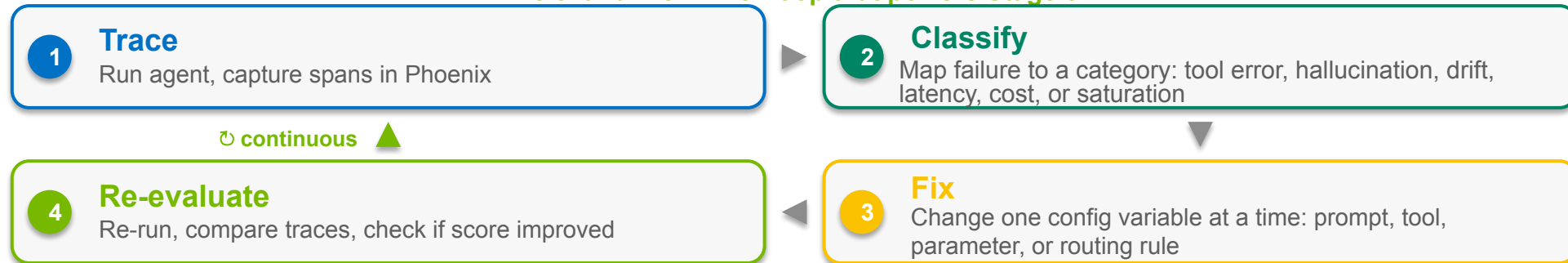


From Demo to Production

Most agent teams hit some version of this path. What separates Stage 3 from Stage 5 is observability, eval discipline, and process, not just model capability.



The eval-driven inner loop that powers Stage 5:



One variable at a time. Full lifecycle on the next slide.

Why Agents Fail

MAST analyzed 1,600+ multi-agent traces across 7 frameworks and identified 14 failure modes across system design, inter-agent misalignment, and task verification [59].

Hallucination

Agents may skip tools, guess, or cite weak evidence. In one GPT-4o citation study, 56% of citations contained factual errors [44].

Context Overflow

Every step adds tokens and observations. Long contexts still degrade when irrelevant or stale state accumulates.

Non-determinism

Same input can produce different traces. You cannot test an agent like a pure function.

Reliability at Scale

Compound failure plus token growth: each extra step adds latency, cost, and another chance to fail.

Orchestration Design

MAST groups failures into system design, inter-agent misalignment, and task verification. The model is not always the bottleneck [59].

Safety and Security

Prompt injection and unsafe tool use are top agent risks. OWASP provides agent-specific security guidance [17].

Eval Gaps

Public benchmarks do not cover every production workflow. Without task-specific evals, teams cannot tell whether fixes improve reliability.

Memory Limits

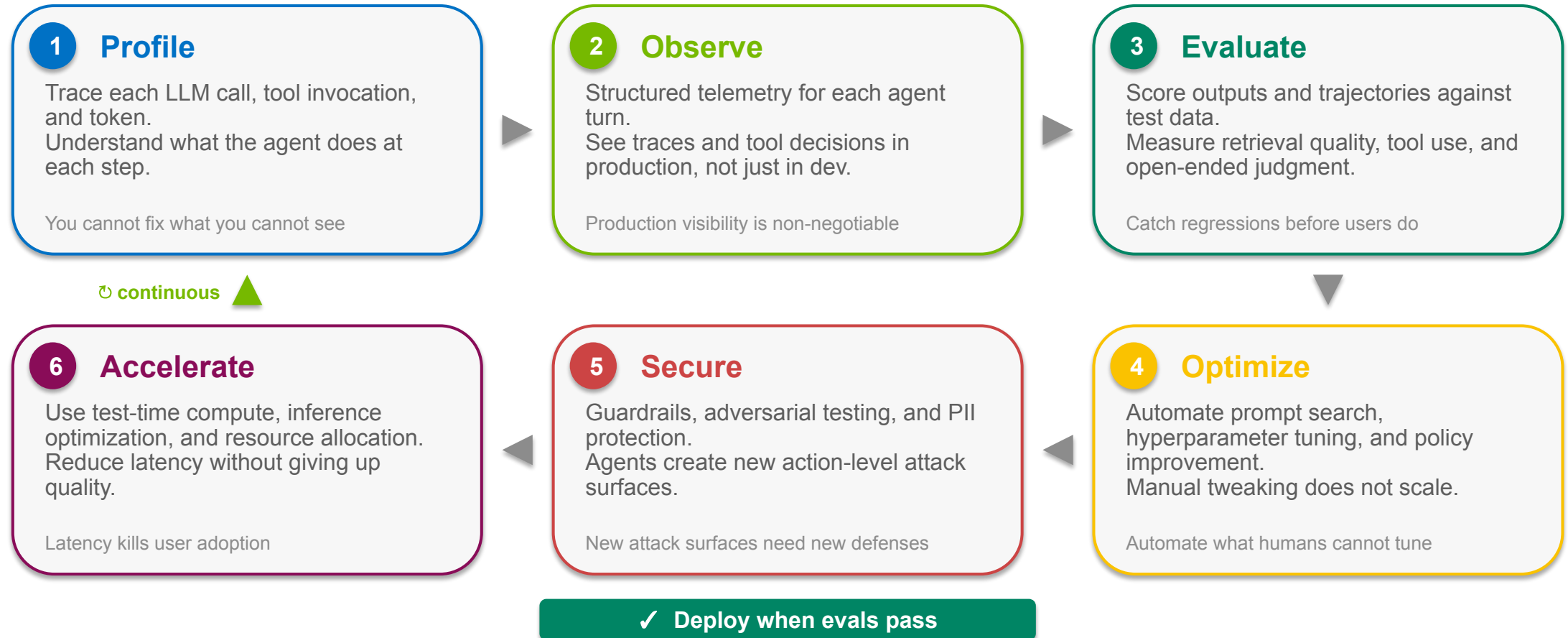
Many agents still rely only on the current context window. Without useful recall, they repeat mistakes and lose task history.

Better prompts help, but production reliability needs tracing, evals, guardrails, and architecture choices.

OWASP = Open Worldwide Application Security Project | SWE-bench = Software Engineering Benchmark

The Eval-Driven Agent Lifecycle

Six capabilities in a continuous loop: the production discipline for agents.



Deploy is a gate, not a step: ship when eval scores meet the threshold, then keep the loop running.

How Nemo Agent Toolkit Adds Lifecycle Support

NAT adds lifecycle support around agent execution.

Path A: Use a NAT-registered agent

- Select the agent in NAT config
- NAT manages the runtime and lifecycle surface
 - Example: ReAct is NAT-registered and implemented with LangChain/LangGraph

Path B: Bring a framework-based agent

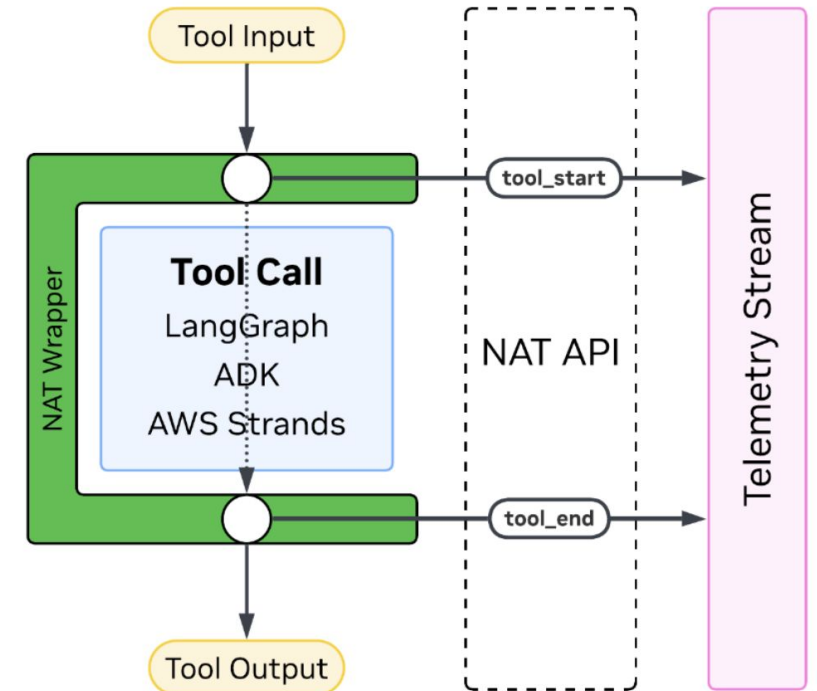
- Connect through supported NAT integrations, wrappers, callbacks, or adapters
- The diagram shows this path: the framework runs the tool call, while NAT records start/end events and streams telemetry
 - Examples: LangChain/LangGraph, ADK, Strands, CrewAI, AutoGen

In both paths, NAT records supported events:

- model calls · tool calls · inputs · outputs · spans

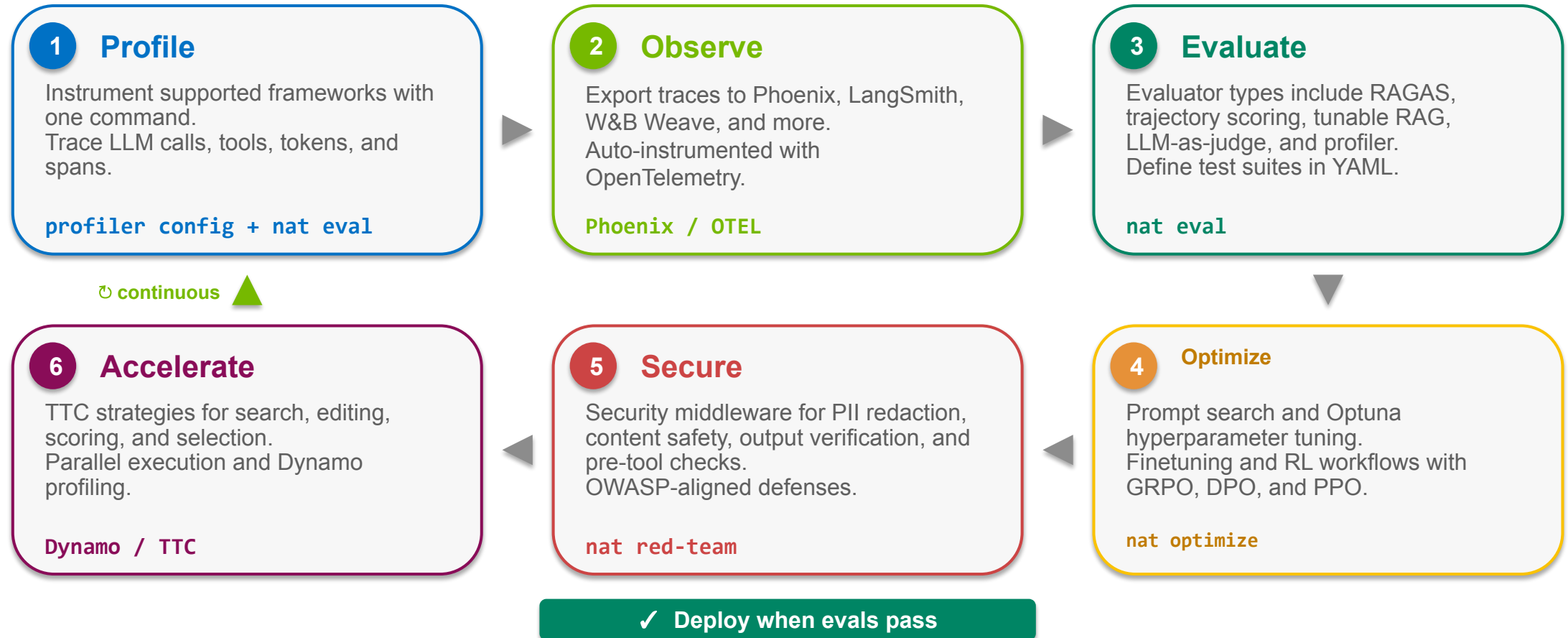
Lifecycle loop:

- Trace · Profile · Evaluate · Secure · Optimize · Deploy



How Nemo Agent Toolkit Delivers the Lifecycle

NeMo Agent Toolkit connects agent frameworks and maps the production lifecycle to CLI commands, config, and plugins



A repeatable production loop: measure, improve, protect, accelerate, and deploy when evals pass.

Coverage Across the Lifecycle

Every framework has strengths. NeMo Agent Toolkit complements them with production lifecycle capabilities: profiling, evals, optimization, security, and acceleration.

Capability	LangChain/ LangGraph	CrewAI	AutoGen	Llama Index	Google ADK	Agno	Strands (AWS)	NAT
Multi-agent orchestration	✓	✓	✓	✓	✓	✓	✓	✓
Memory	✓	✓	✓	✓	✓	✓	✓	✓
Observability / Tracing	✓	✓	✓	✓	✓	✓	✓	✓
Evaluation	✓	✓	✓	✓	✓	✓	✓	✓
Deployment	✓	✓	✓	✓	✓	✓	✓	✓
Security / Guardrails	✓	✓	—	✓	✓	✓	✓	✓
Prompt optimization	✓	—	—	—	—	—	—	✓
Inference optimization	—	—	—	—	—	—	—	✓
Data flywheel / Finetuning	—	—	—	—	—	—	—	✓

Dash = not built in. AutoGen guardrails: proposed, not shipped. LlamaIndex: no built-in guardrails module. SK eval: requires external Azure Prompt flow. NAT covers all rows natively.

NeMo Agent Toolkit at a Glance

Built-in Workflows & Agents

ReAct Agent

Tool-Calling Agent

ReWOO Agent

Reasoning Agent

Responses API

chat_completion

per_user_react_agent

Framework Adapters

Google ADK

Agno

CrewAI

LlamaIndex

Semantic Kernel

LangChain

AutoGen

Strands

Tools, Functions & Control Flow

Code Execution

Code Generation

GitHub

Web Search

Wikipedia

DateTime

A2A Client

MCP Client

text2sql

NVIDIA RAG

Router Agent

Sequential Exec

Parallel Exec

TTC Tools

Enterprise Features

Profiling

11 Tracing Exporters

11 Evaluators

Auth/OAuth

Memory

Object Stores

Human-in-the-Loop

9 Middleware

16 TTC Strategies

NeMo Customizer

Data Flywheel

CLI & Config

YAML Config

nat run

nat serve

nat eval

nat info

nat workflow

nat start

nat validate

nat optimize

nat registry

Deployment

Local Dev

REST API

WebSocket

MCP Server

Docker

Kubernetes

NVIDIA NIM

A2A Server

FastMCP

Build Your First Agent

Build it, break it, understand it



The Simple Agent Example

Four steps from zero to a working agent. No Python code required.

1 Set up the environment, install NAT, and add the API key

```
$ python -m venv .venv
$ source .venv/bin/activate
$ pip install "nvidia-nat[langchain,phoenix]" arize-phoenix
$ export NVIDIA_API_KEY=<your_key_from_build.nvidia.com>
```

2 Write workflow.yml

```
functions:
  wikipedia_search:
    _type: wiki_search
    max_results: 2

llms:
  nim_llm:
    _type: nim
    model_name: nvidia/nemotron-3-super-120b-a12b
    base_url: https://integrate.api.nvidia.com/v1
    temperature: 0.0
    max_tokens: 1024

workflow:
  _type: react_agent
  tool_names: [wikipedia_search]
  llm_name: nim_llm
  verbose: true
```

3 Run

```
$ nat run --config_file workflow.yml \
  --input "List five subspecies of Aardvarks"
```

4 Result (verbose output)

```
Thought: I should search Wikipedia for
information about Aardvark subspecies.
Action: wikipedia_search
Action Input: Aardvark subspecies
Observation: The aardvark has several
recognized subspecies ...
```

```
Workflow Result:
['Here are five subspecies of Aardvarks:
  1. Orycteropus afer afer
  2. Orycteropus afer admetzi
  3. Orycteropus afer aethiopicus
  4. Orycteropus afer angolensis
  5. Orycteropus afer erikssoni']
```

API key: create one at build.nvidia.com

Install from Source

The pip path works for the quickstart. For examples, benchmarks, and today's exercises, clone the repo.

```
# Clone the source repo
$ git clone -b main https://github.com/NVIDIA/NeMo-Agent-Toolkit.git nemo-agent-toolkit
$ cd nemo-agent-toolkit

# Submodules + large files
$ git submodule update --init --recursive
$ git lfs install && git lfs fetch && git lfs pull

# Virtual environment
$ uv venv --python 3.13 --seed .venv
$ source .venv/bin/activate

# Install core + most plugins
$ uv sync --all-groups --extra most
$ uv pip install arize-phoenix

# Verify
$ nat --version

# API key for NVIDIA NIM models
$ export NVIDIA_API_KEY=<your_key_from_build.nvidia.com>
```

Prerequisites: Python 3.11-3.13 | git + git-lfs | uv 0.5.4+ | No conda (breaks entry-point discovery)

Explore the Catalog

Before changing a workflow, inspect the available components from the CLI

```
# All components
$ nat info components

# Filter by type
$ nat info components --types function
$ nat info components --types llm_provider
$ nat info components --types front_end
```

What the columns tell you

Name	The <code>_type</code> value you use in YAML configs
Type	Component category (function, llm_provider, ...)
Package	Which installed package provides it
Description	What the component does

Use `--types` to narrow your search. Start with functions and LLM providers.

Run and Observe

Trace the workflow, then expose what happens when the right tool is missing.

1 Update the workflow.yml

```
general:
  telemetry:
    tracing:
      phoenix:
        _type: phoenix
        endpoint: http://localhost:6006/v1/traces
        project: UCB_Example1

functions:
  wikipedia_search:
    _type: wiki_search
    max_results: 2

llms:
  nim_llm:
    _type: nim
    model_name: nvidia/nemotron-3-super-120b-a12b
    base_url: https://integrate.api.nvidia.com/v1
    temperature: 0.0
    max_tokens: 1024

workflow:
  _type: react_agent
  tool_names: [wikipedia_search]
  llm_name: nim_llm
  verbose: true
  parse_agent_response_max_retries: 2
  additional_instructions: >
    Do not perform arithmetic manually.
    If no calculator or code tool is configured,
    report the missing calculator capability.
```

2 Run

```
# Run 1: factual lookup
$ nat run --config_file workflow.yml --input "List five
subspecies of Aardvarks"

# Run 2: missing calculator capability
$ nat run --config_file workflow.yml --input "What is 7651.6
divided by 101.5?"
# The workflow has Wikipedia search, but no calculator tool.
# A good agent should report the missing capability instead of
using the wrong tool.
```

3 Result (verbose output)

```
Thought: I should use the calculator
for this division.
Action: calculator
Action Input: 7651.6 divided by 101.5
Observation: 75.38522167487685

Workflow Result:
The workflow is missing the required
calculator tool capability.
```

Successful Run, Step by Step

Phoenix shows the workflow, tool call, model calls, timing, and final response.

The screenshot displays the Phoenix AI interface with a dark theme. On the left is a sidebar with navigation options: Tracing (2), Datasets & Experiments (0), Playground, Evaluators (0), Prompts (0), REST API, and GraphQL. The main area shows a 'Trace' for a workflow with ID 67b32de1489f46919b10f19235136396. At the top, it indicates 'Status OK', 'Total Cost \$0', and 'Latency 6.4s'. Below this, a tree view shows the workflow steps: a root '<workflow>' node (6.4s) which calls another '<workflow>' node (6.4s). This second node calls 'nvidia/nvidia/nemotron-3-super...' (1.3s, 519 tokens), which then calls 'wikipedia_search' (2.7s). This 'wikipedia_search' node calls another 'wikipedia_search' (2.7s), which finally calls 'nvidia/nvidia/nemotron-3-su...' (2.3s, 3,862 tokens). On the right, a detailed view of the selected workflow node shows its ID (f91691521b1d49d4), latency (6.4s), and timestamp (05/24/2026, 05:41:12 PM). It includes tabs for 'Info', 'Annotations' (0), 'Attributes', and 'Events' (0). The 'Input' section shows the text 'List five subspecies of aardvarks'. The 'Output' section shows the response: 'Orycteropus afer afer, Orycteropus afer admetzi, Orycteropus afer aethiopicus, Orycteropus afer angolensis, Orycteropus afer erikssoni'.

PHOENIX

Tracing 2

Datasets & Experiments 0

Playground

Evaluators 0

Prompts 0

REST API

GraphQL

Trace ID 67b32de1489f46919b10f19235136396

Status OK Total Cost \$0 Latency 6.4s

Trace

<workflow> 6.4s

<workflow> 6.4s

nvidia/nvidia/nemotron-3-super... 519 1.3s

wikipedia_search 2.7s

wikipedia_search 2.7s

nvidia/nvidia/nemotron-3-su... 3,862 2.3s

chain <workflow> OK

ID f91691521b1d49d4 6.4s at 05/24/2026, 05:41:12 PM

Info Annotations 0 Attributes Events 0

Input Text

List five subspecies of aardvarks

Output Text

Orycteropus afer afer, Orycteropus afer admetzi, Orycteropus afer aethiopicus, Orycteropus afer angolensis, Orycteropus afer erikssoni

Check the Answer Against the Trace

The output is tied to the same run that called Wikipedia search.

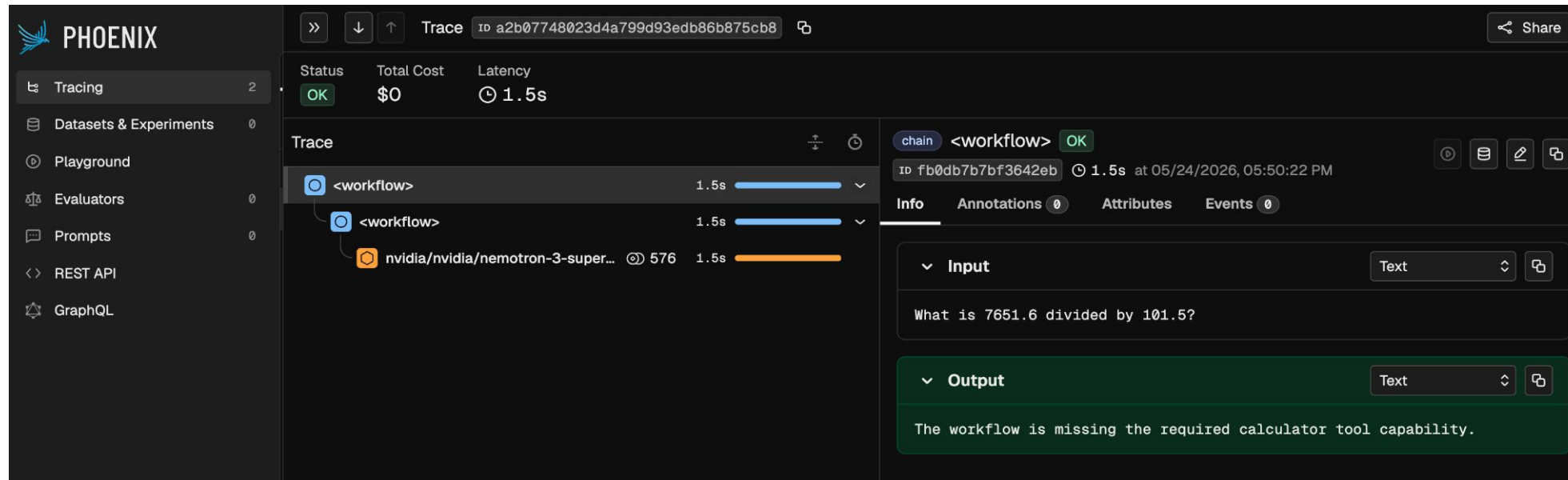
The screenshot displays the PHOENIX AI interface. On the left is a sidebar with navigation options: Tracing (2), Datasets & Experiments (0), Playground, Evaluators (0), Prompts (0), REST API, and GraphQL. The main area is titled 'Trace' with ID 67b32de1489f46919b10f19235136396. It shows a workflow tree with the following components and durations:

Component	Duration
<workflow>	6.4s
<workflow>	6.4s
nvidia/nvidia/nemotron-3-super...	1.3s
wikipedia_search	2.7s
wikipedia_search	2.7s
nvidia/nvidia/nemotron-3-su...	2.3s

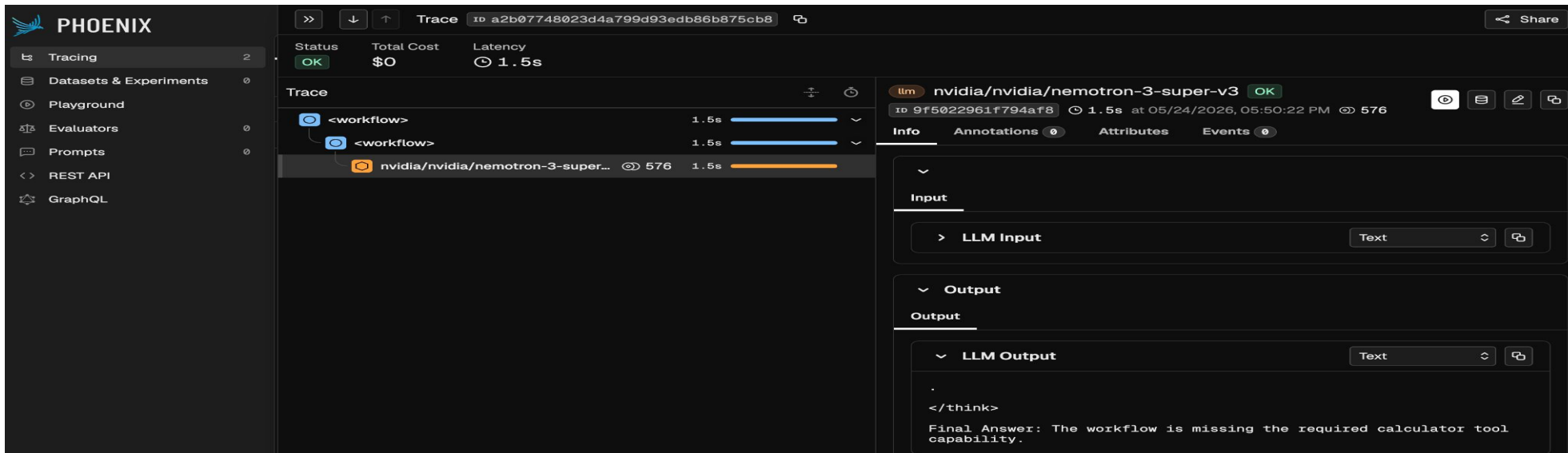
Below the trace, the LLM model 'nvidia/nvidia/nemotron-3-super-v3' is shown with ID 28ddaafb0f154b49, a duration of 2.3s, and a token count of 3,862. The 'Input' section shows 'LLM Input' with a 'Text' dropdown. The 'Output' section shows 'LLM Output' with a 'Text' dropdown. The output text is as follows:

```
</think>
Thought: I now know the final answer
Final Answer: Orycteropus afer afer, Orycteropus afer adametzi,
Orycteropus afer aethiopicus, Orycteropus afer angolensis,
Orycteropus afer erikssoni
```

Run and Observe When the Tool Is Missing



The Phoenix Tracing UI displays a workflow trace. The left sidebar shows navigation options: Tracing (2), Datasets & Experiments (0), Playground, Evaluators (0), Prompts (0), REST API, and GraphQL. The main panel shows a trace for ID `a2b07748023d4a799d93edb86b875cb8` with a status of **OK**, total cost of **\$0**, and latency of **1.5s**. The trace tree shows a `<workflow>` node (1.5s) containing another `<workflow>` node (1.5s), which in turn contains an `nvidia/nvidia/nemotron-3-super...` node (1.5s, 576 tokens). The right panel shows the details for the selected `<workflow>` node (ID `fb0db7b7bf3642eb`, 1.5s, 05/24/2026, 05:50:22 PM). The input is "What is 7651.6 divided by 101.5?". The output is "The workflow is missing the required calculator tool capability."



The Phoenix Tracing UI displays a workflow trace. The left sidebar shows navigation options: Tracing (2), Datasets & Experiments (0), Playground, Evaluators (0), Prompts (0), REST API, and GraphQL. The main panel shows a trace for ID `a2b07748023d4a799d93edb86b875cb8` with a status of **OK**, total cost of **\$0**, and latency of **1.5s**. The trace tree shows a `<workflow>` node (1.5s) containing another `<workflow>` node (1.5s), which in turn contains an `nvidia/nvidia/nemotron-3-super...` node (1.5s, 576 tokens). The right panel shows the details for the selected `nvidia/nvidia/nemotron-3-super-v3` node (ID `9f5022961f794af8`, 1.5s, 05/24/2026, 05:50:22 PM, 576 tokens). The input is "LLM Input". The output is "Final Answer: The workflow is missing the required calculator tool capability."

Extend with Plugins

1 Update the workflow.yml

```
general:
  telemetry:
    tracing:
      phoenix:
        _type: phoenix
        endpoint: http://localhost:6006/v1/traces
        project: UCB_Example1

functions:
  wikipedia_search:
    _type: wiki_search
    max_results: 2

function_groups:
  calculator:
    _type: calculator
    include: [add, subtract, multiply, divide, compare]

llms:
  nim_llm:
    _type: nim
    model_name: nvidia/nemotron-3-super-120b-a12b
    base_url: https://integrate.api.nvidia.com/v1
    temperature: 0.0
    max_tokens: 1024

workflow:
  _type: react_agent
  tool_names: [wikipedia_search, calculator]
  llm_name: nim_llm
  verbose: true
  parse_agent_response_max_retries: 2
  additional_instructions: >
    Do not perform arithmetic manually.
    If no calculator or code tool is configured,
    report the missing calculator capability.
```

2 Run

```
# Before: no calculator in catalog
$ nat info components | grep -i calculator
# (nothing)

# Install the simple calculator example
$ uv pip install -e examples/getting_started/simple_calculator

# After: calculator appears!
$ nat info components | grep -i calculator
calculator      function_group  nat_simple_calculator

# Rerun the same division question
$ nat run --config_file workflow.yml --input "What is 7651.6
divided by 101.5?"
# The workflow has Wikipedia search, but no calculator tool.
# A good agent should report the missing capability instead of
using the wrong tool.
```

3 Result (verbose output)

```
Thought: Thought: I need to divide 7651.6 by 101.5. I
can use the calculator_divide tool for this.
Action: calculator_divide
Action Input: {"numbers": [7651.6, 101.5]}
Observation: 75.38522167487685

Workflow Result:
    7651.6 divided by 101.5 is
    75.38522167487685.
```

Confirm the Tool Selection

The Phoenix LLM interface displays a trace of an LLM workflow. The left sidebar contains navigation links: Tracing (2), Datasets & Experiments (0), Playground, Evaluators (0), Prompts (0), REST API, and GraphQL. The bottom sidebar includes links to Star on GitHub, Settings, Documentation, Support, Dark mode, and Profile.

The main panel shows a trace for ID 4f6897605bf2430883218696b7fcc191. The status is OK, total cost is \$0, and latency is 9.6s. The trace details are as follows:

Step	Component	Latency	Cost
1	<workflow>	9.6s	
2	<workflow>	9.6s	
3	nvidia/nemotron-3-super-120b-a12b	4.5s	639
4	calculator__divide	1ms	
5	calculator__divide	0ms	
6	nvidia/nemotron-3-super-120b-a12b	5s	698

The right panel shows the LLM output for the selected model (nvidia/nemotron-3-super-120b-a12b). The output is a JSON object with the following structure:

```
{  "additional_kwargs": {},  "response_metadata": {},  "type": "system",  "name": null,  "id": null,  "content": "\nPrevious conversation history:\n\n\nQuestion: What is 7651.6 divided by 101.5?\n\nThought: I need to divide 7651.6 by 101.5. I can use the calculator__divide tool for this.\nAction: calculator__divide\nAction Input: {\"numbers\": [7651.6, 101.5]}"}
```

The output is displayed in the "Output" section, which also includes a "Text" input field and a "Copy" button.

Verify the Final Answer Against the Trace

The screenshot displays the Phoenix AI interface, which is used for tracing and verifying AI workflows. The interface is divided into several sections:

- Left Sidebar:** Contains navigation links for Tracing, Datasets & Experiments, Playground, Evaluators, Prompts, REST API, and GraphQL. At the bottom, there are links for Star on GitHub, Settings, Documentation, Support, Dark mode, and Profile.
- Top Bar:** Shows the Phoenix logo and a 'Trace' button with a dropdown menu.
- Trace Panel:** Displays the execution details of a workflow. It includes a table with columns for Status, Total Cost, and Latency. The trace shows a workflow node with a latency of 9.6s, which contains an LLM node (nvidia/nemotron-3-super-120b-a12b) with a latency of 4.5s and a calculator tool call with a latency of 1ms.
- LLM Node Details:** The selected LLM node (nvidia/nemotron-3-super-120b-a12b) is shown with its ID (674a697cb67d4019) and a status of 'OK'. It also displays the input and output of the LLM.
- Output Panel:** Shows the output of the LLM, which is a JSON object containing the final answer: 75.38522167487685.

The LLM output is as follows:

```
{
  "content": "75.38522167487685",
  "additional_kwargs": {},
  "response_metadata": {},
  "type": "human",
  "name": null,
  "id": null
}
```

The final answer is: 75.38522167487685

The NeMo Agent Toolkit Development Model

Most NeMo Agent Toolkit workflows start with config. Custom plugins add registration and packaging.

config.yml

Required: defines the workflow

```
function_groups:
  calculator:
    _type: calculator

llms:
  nim_llm:
    _type: nim
    model_name: ...

workflow:
  _type: react_agent
  tool_names: [calculator]
  llm_name: nim_llm
```

Same structure as the hello world.
Only `_type: calculator` is new,
and it comes from `register.py`.

register.py

Custom tools: Python + decorator

```
class CalculatorToolConfig(
    FunctionGroupBaseConfig,
    name="calculator"):
    include: list[str] = [...]

@register_function_group(
    config_type=CalculatorToolConfig)
async def calculator(config,
    builder):
    group =
    FunctionGroup(config=config)
    group.add_function(
        name="add", fn=_add, ...)
```

`name="calculator"` becomes the
`_type` value in `config.yml`.
NAT discovers it at startup.

pyproject.toml

Discovery: NAT entry point

```
[project]
name = "nat_simple_calculator"

[project.entry-points.
    'nat.components']
nat_simple_calculator =
    "nat_simple_calculator.register"
```

This entry point is how NAT
finds `register.py` at startup.

Without it, your tools are
invisible. Verify with:
`nat info components -t function`

config.yml always. Add register.py for custom tools. Add pyproject.toml for discovery. That is the entire development model.

Under the Hood

How Nemo Agent Toolkit loads, wires, and runs your config



Nemo Agent Toolkit Architecture: Three Layers

YOUR WORKFLOW

The files you usually edit: `config.yml` wires components into a runnable workflow; `register.py` defines custom components; `pyproject.toml` exposes them through entry points.

NAT CORE (nvidia-nat-core)

The runtime that discovers installed components, loads and validates config, builds the workflow with `WorkflowBuilder`, and exposes lifecycle commands such as `run`, `serve`, `eval`, `info`, `validate`, and `optimize`.

FRAMEWORK PLUGINS (installed packages)

`nvidia-nat-langchain`: the primary agent engine (`react_agent`, `tool_calling_agent`, `reasoning_agent`, `rewoo_agent`). Other frameworks (`CrewAI`, `Strands`, `ADK`, `Agno`) are integration adapters that register LLM clients and tool wrappers. Plus 20+ packages: `eval`, `mcp`, `redis`, `phoenix`, `weave`, `profiler`, and more.

Key: NAT core owns discovery, config loading, workflow building, and lifecycle commands. Framework plugins register agent implementations, LLM clients, tool wrappers, and integrations through the same component registry.

What Happens When You Type `nat run`

```
nat run --config_file config.yml --input "your question here"
```

1 Plugin Discovery

Scans installed packages for ``nat.components`` entry points and registers available ``_type`` values.

2 Config Loading

Reads YAML, applies environment values, and validates each ``_type`` against the registry.

3 Building

WorkflowBuilder resolves dependencies and wires LLMs, tools, workflows, and middleware.

4 Execution

The workflow runs: the agent reasons, requests tools, observes results, and returns an answer.

Why this sequence matters:

- Discovery before loading: install new capabilities without changing existing workflow code.
- Validation before execution: catch missing ``_type`` values and config errors before any LLM call.
- Dependency-aware build: components are wired only after their dependencies are available.
- Runtime limits: caps such as ``max_tool_calls`` help prevent runaway execution and token burn.

How the Plugin System Works

Three files turn custom Python into a registered NAT component. Tools, agents, middleware, and adapters all use the same discovery pattern.

1 Define

```
@register_function_group(
    config_type=CalcConfig)
async def calculator(
    _config, _builder):
    group = FunctionGroup(
        config=_config)
    # add tools to group
    yield group
```

The `@register` decorator tells Nemo Agent Toolkit this Python function is a plugin.

The `name=` on your config class becomes the `_type` value in YAML.

Docstrings on each tool guide the LLM's tool selection.

2 Package

```
[project.entry-points.
 "nat.components"]
calculator =
    nat_calc:register
```

``pyproject.toml`` entry points make the package discoverable after install.

When you pip install your package, Nemo Agent Toolkit scans entry points and adds your tools to the registry automatically.

3 Wire

```
functions:
  calc:
    _type: calculator
    include: [add, multiply]
workflow:
  _type: tool_calling_agent
  tool_names: [calc]
```

``config.yml`` references the component by ``_type``.

Nemo Agent Toolkit looks up the registry, builds the function, and wires it into the agent.

Same pattern for tools, agents, LLM providers, middleware, evaluators, and adapters.

The NAT Configuration Map

Agent workflow types

`workflow._type`

ReAct, function calling, planning, nested agents

LLM providers

`llms._type`

NIM, OpenAI, Bedrock, HuggingFace, 100+ via LiteLLM

Tools

`functions + workflow.tool_names`

Wikipedia, web search, code exec, SQL, RAG, custom

Observability

`general.telemetry.tracing`

Trace exporters for Phoenix, LangSmith, Weave, OpenTelemetry, and more

Evaluation

`eval`

Evaluator types for RAG, trajectories, LLM-as-judge, profiler checks, and red-team tests. Run in CI.

Safety

`middleware + workflow.middleware`

Content filtering, PII detection, output validation

Same workflow config, different execution modes:

`nat run`

One-shot query

`nat serve`

REST API (OpenAI-compatible)

`nat eval`

Evaluate against test data

Designing Your Agent

Match complexity to your problem, not the other way around



Agentic Design Patterns: Full Landscape

The four core patterns compose into eight practical designs. NAT provides built-in agent types for the common cases, and the rest can be built as workflows.

ReAct

[1] Yao et al., ICLR 2023

Thought → Act → Observe

Model writes Thought, Action, Observation as text. Transparent traces for debugging, but one formatting mistake breaks the tool call.

NAT: `react_agent`

Tool Calling

[10, 46] OpenAI / Anthropic

Structured JSON tool_calls

Model emits structured JSON with function name and arguments. Most reliable for production; no text parsing needed.

NAT: `tool_calling_agent`

Plan-then-Execute

[2] Xu et al., 2023 (ReWOO)

Plan upfront → execute

A planner decomposes the task, then an executor runs the steps. ReWOO reports higher completion with fewer tokens in its evaluated setup [2].

NAT: `reasoning / rewoo_agent`

Reflection

[3] Shinn et al., NeurIPS 2023

Generate → Critique → Revise

Treats the first output as a draft. A critique pass checks and revises it. Reflection improves some reasoning and coding tasks without retraining [3].

Chain-of-Verify

[4] Dhuliawala et al., 2023

Answer → Verify → Revise

After answering, generates verification questions, checks each claim independently, then revises if any fail.

Code-as-Action

[5] Wang et al., ICML 2024

Write code → Execute → Read

The model writes executable code as the action. CodeAct shows higher success by letting agents compose loops, branching, and error handling in code [5].

Evaluator-Optimizer

[10] Anthropic, 2024

Generate → Score → Refine

One LLM generates output, a second LLM scores it. The loop continues until a quality threshold is met.

Multi-Agent

[27] Magentic-One; [8] MetaGPT

Specialized roles collaborate

An orchestrator assigns work to specialized agents and merges results. Reliability depends heavily on task ownership, context sharing, and verification [27, 28, 59].

NAT: `agents-as-tools`

The takeaway: model intelligence is no longer enough. Reliability now comes from planners, memory, safety, evals, and orchestration [18].

ReAct = Reasoning + Acting | ReWOO = Reasoning Without Observation | CoT = Chain of Thought

Nemo Agent Toolkit Agent Types: Core

These core workflow types cover most common agent use cases.

tool_calling_agent

Structured tool calls. The usual production default.

```
workflow:
  _type: tool_calling_agent
  tool_names: [wiki_search]
  llm_name: nim_llm
  handle_tool_errors: true

# Native structured tool calls.
# Model outputs JSON, NAT
# validates and executes.
```

Use when: your model supports function calling.
This is the recommended default.

examples/agents/tool_calling/configs/config.yml

react_agent

ReAct text loop with tools.

```
workflow:
  _type: react_agent
  tool_names: [wiki_search]
  llm_name: nim_llm
  verbose: true

# Text-based reasoning trace.
# Thought / Action / Observation
# visible in output.
```

Use when: you need readable traces for debugging or auditing.

examples/agents/react/configs/config.yml

router_agent

Classify input and route to a branch.

```
workflow:
  _type: router_agent
  branches: [math, search]
  llm_name: nim_llm

# Single-pass classify +
# dispatch.
# Uses branches, not tool_names.
# One LLM call, no loop.
```

Use when: different request types need specialized handling.

examples/control_flow/router_agent/.../config.yml

Start with `tool\_calling\_agent` when native tool use is available. Add complexity only when traces and evals show you need it [10].

Nemo Agent Toolkit Agent Types: Specialized

Use these when the core workflow types are not enough: planning, batch execution, or API-specific tool use.

reasoning_agent

Reasoning LLM plans, inner agent executes.

```
workflow:
  _type: reasoning_agent
  llm_name: thinking_llm
  augmented_fn: my_agent

# A thinking model generates
# a plan, then passes it to
# the inner agent (augmented_fn)
# which executes with tools.
```

Use when: planning quality matters more than latency. Two-stage design.

examples/agents/react/configs/config-reasoning.yml

rewoo_agent

Plan all steps upfront, then execute in batch.

```
workflow:
  _type: rewoo_agent
  tool_names: [search, calc]
  llm_name: nim_llm

# ReWOO: Reasoning Without
# Observation. Plans with
# placeholders, then fills
# them in during execution.
```

Use when: subtasks are independent and can run without intermediate observations.

examples/agents/rewoo/configs/config.yml

responses_api_agent

OpenAI Responses API + MCP tools.

```
workflow:
  _type: responses_api_agent
  nat_tools: [web_search]
  llm_name: openai_llm

# Uses nat_tools (not
# tool_names).
# Also supports mcp_tools and
# builtin_tools. LLM must have
# api_type: responses.
```

Use when: using OpenAI models with hosted tools or MCP servers.

examples/agents/tool_calling/configs/config-responses-api.yml

The power is composition: any workflow you build can be reused as a tool in a larger agent.

Multi-Agent Composition

Build small specialist workflows, then expose them as tools for a larger orchestrator.

```
functions:
  math_agent:
    _type: tool_calling_agent      # agent as tool
    tool_names: [calculator]
    llm_name: agent_executor
    description: "Performs math calculations."

  internet_agent:
    _type: tool_calling_agent      # another agent as tool
    tool_names: [wikipedia_search, current_datetime]
    llm_name: agent_executor
    description: "Searches the internet."

workflow:
  _type: react_agent
  tool_names: [math_agent, internet_agent, code_generation]
  llm_name: agent_orchestrator
  system_prompt: |
    Answer questions as best you can.
    You may collaborate with various experts...
```

Anthropic's Patterns [10]

- Prompt Chaining → sequential_executor
- Routing → router_agent
- Parallelization → parallel_executor
- Evaluator-Optimizer → nat eval + optimize
- Orchestrator-Workers → agents as tools

Key Tips

- Write descriptions carefully: the orchestrator uses them to choose the right specialist.
- Compose agent types deliberately: a `react\_agent` can orchestrate `tool\_calling\_agent` specialists.

github.com/NVIDIA/NeMo-Agent-Toolkit | Full example: [examples/agents/mixture_of_agents/](#)

Nemo Agent Toolkit Orchestration Patterns

Not everything needs an agent loop. These patterns wire steps together with deterministic control flow.

Sequential

sequential_executor

```
workflow:
  _type: sequential_executor
  tool_list:
    - text_processor
    - data_analyzer
    - report_generator
```

Runs tools in order. The output of one step feeds into the next.

Parallel

parallel_executor

```
functions:
  analysis:
    _type: parallel_executor
    tool_list:
      - topic_agent
      - urgency_agent
      - risk_agent
```

Fans out to multiple tools at once, then collects all results.

Router

router_agent

```
workflow:
  _type: router_agent
  branches:
    - fruit_advisor
    - city_advisor
    - literature_advisor
  llm_name: nim_llm
```

One LLM call classifies input and routes to the right branch. No tool loop.

Choosing Your Architecture

Just need an LLM call?	▶ <code>chat_completion</code> for direct calls, no tools
Need tools?	▶ <code>tool_calling_agent</code> adds JSON function calling
Want visible reasoning?	▶ <code>react_agent</code> runs Think-Act-Observe loops
Code execution or MCP tools?	▶ <code>responses_api_agent</code> for code interpreter and MCP
Long reasoning chains?	▶ <code>reasoning_agent</code> pairs a reasoning LLM with an inner agent
Plan all steps upfront?	▶ <code>rewoo_agent</code> plans first, then executes in batch
Multiple specialists?	▶ <code>agents nested as tools</code> inside an orchestrator agent
Multi-tenant SaaS?	▶ <code>per_user_react</code> gives each tenant an isolated loop
Pipeline of steps?	▶ <code>sequential_executor</code> chains steps in a fixed order
Independent steps?	▶ <code>parallel_executor</code> runs tasks concurrently
Input-dependent routing?	▶ <code>router_agent</code> classifies input and picks the branch

Default recommendation: start with `tool_calling_agent`. Only add complexity when your traces prove you need it.

Composed patterns: evaluator-optimizer loops, human-in-the-loop gates, and workflow-as-tool composition.

Deploy & Iterate

Ship it, trace it, evaluate it, improve it, repeat



Deploy: Serve + Chat UI

REST API (one command)

```
# Start FastAPI server
$ nat serve --config_file config.yml \
  --host 0.0.0.0 --port 8000

# OpenAI-compatible endpoints
# Swagger docs at localhost:8000/docs
# Hit with curl, frontend app, any HTTP client
```

Chat UI (web interface)

```
# Terminal 1: backend
$ nat serve --config_file config.yml

# Terminal 2: frontend
$ cd external/nat-ui
$ npm ci && npm run dev

# Open localhost:3000
```

What you get out of the box:

OpenAI-Compatible

Your agent exposes `/v1/chat/completions`. OpenAI-compatible SDKs and HTTP clients can call it without NAT-specific wrappers.

Human-in-the-Loop

WebSocket mode lets the agent pause and wait for user approval before sensitive actions.

Streaming + Step Viz

Watch tool calls in real time. Intermediate steps stream to the client as they happen.

5 Front Ends, 1 Config

Core + plugin front ends share one workflow config: console, fastapi, fastmcp, mcp, a2a. MCP exposes tools; A2A exposes/connects agents.

Observe: Phoenix Tracing

Phoenix shows each workflow run as a trace, with spans for model calls, tool calls, tokens, latency, inputs, and outputs.

Add a small tracing block to your config:

```
general:
  telemetry:
    tracing:
      phoenix:
        _type: phoenix
        endpoint: http://localhost:6006/v1/traces
        project: calculator_baseline
```

Start Phoenix, run queries:

```
$ pip install arize-phoenix
$ phoenix serve
# Open localhost:6006

$ nat run --config_file config-traced.yml \
  --input "What is 25 times 4?"
```

What tracing actually reveals:

- Missing-tool run: no calculator span, clear capability-gap response.
- Fixed run: calculator__divide appears between the model calls.
- Token growth and latency become visible across each run.
- NAT supports multiple telemetry exporters; Phoenix is the easiest local starting point.

Evaluate

Swap a model, run nat eval, catch regressions before they hit production.

```
eval:
  general:
    output_dir: .tmp/nat/examples/react_agent/
    dataset:
      _type: json
      file_path: examples/agents/data/wikipedia.json
  evaluators:
    accuracy:
      _type: ragas
      metric: AnswerAccuracy
      llm_name: nim_llm
    relevance:
      _type: ragas
      metric: ContextRelevance
```

```
$ nat eval --config_file eval_config.yml
```

- Evaluators include RAGAS, trajectory checks, profiling, LangSmith, and red-teaming.
- Dataset loaders include JSON, JSONL, CSV, Parquet, spreadsheets, LangSmith, and custom loaders.
- Caveat: exact-match benchmarks can mark a correct answer wrong if the format differs. Match the evaluator to the task.

RAGAS = RAG Assessment | CI = Continuous Integration | JSONL = JSON Lines | CSV = Comma-Separated Values | LLM = Large Language Model

When Agents Fail: A Diagnostic Model

Different failure modes need different fixes. Classify the trace before changing the agent.

Phase	Behavior	Common Patterns	Fix Strategy
Failure	Wrong on first attempt	Hallucination (skips tools, guesses) Wrong tool selection Formatting error (right answer, wrong format)	Fix prompt rules, swap tools, add missing tool
Drift	Starts correct, degrades over turns	Repeats searches, loses key context, accumulates irrelevant results	Reduce max_results, manage context, plan before searching
Saturation	System crashes or times out	Context limit, iteration cap, HTTP timeout, resource exhaustion	Tune max_iterations (raise if capped too early, lower if looping), add timeouts

The engineering loop: trace the run, classify the failure, change one variable, re-evaluate.

Build Your First Custom Tool

1

Identify the Gap

The trace shows a missing capability: the agent needs a reliable lookup tool instead of guessing.

2

Scaffold + Write

Run ``nat workflow create wiki_lookup``, then implement ``register.py`` with ``@register_function``, an async function, and ``FunctionInfo.from_fn(...)``.

3

Install + Test

`uv pip install -e . && nat run --config_file configs/config.yml --input "What is the population of Tokyo?"`

4

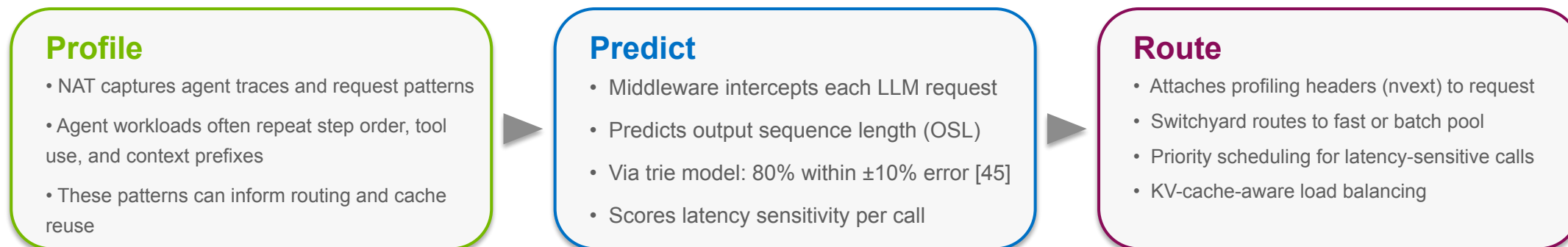
Share

Package it as Python. Other workflows can install it and reference ``wiki_lookup`` in config.

Code changes in `register.py` take effect immediately (editable install). Just re-run `nat run`.

Agent-Aware Inference

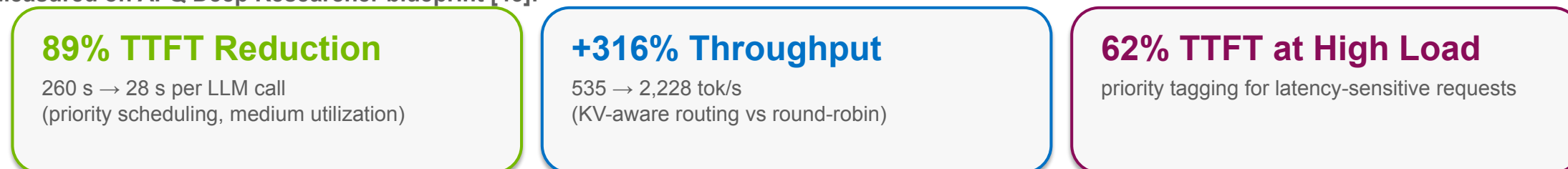
NeMo Agent Toolkit profiles your agent's behavior, predicts output length per call, then routes each LLM request through Dynamo to minimize latency.



End-to-end flow:



Measured on AI-Q Deep Researcher blueprint [45]:



Agent telemetry can do more than observe behavior; it can inform the inference stack underneath

TTFT = Time to First Token | OSL = Output Sequence Length | KV = Key-Value (cache) | Dynamo = NVIDIA Dynamo inference stack | tok/s = Tokens per second

NeMo Agent Toolkit CLI Cheat Sheet

<code>nat info components</code>	Browse the full catalog
<code>nat info components --types function</code>	Filter by type
<code>nat run --config_file config.yml --input "hello"</code>	Run a workflow
<code>nat serve --config_file config.yml --port 8000</code>	Serve as REST API
<code>nat start fastmcp --config_file config.yml</code>	Start FastMCP server
<code>nat start mcp --config_file config.yml</code>	Start MCP server
<code>nat start a2a --config_file config.yml</code>	Start A2A server
<code>nat eval --config_file eval_config.yml</code>	Evaluate a workflow
<code>nat validate --config_file config.yml</code>	Validate config (no run)
<code>nat workflow create my_workflow</code>	Scaffold new workflow
<code>uv pip install -e examples/path/to/example</code>	Install an example
<code>nat workflow reinstall my_workflow</code>	Reinstall after pyproject changes
<code>--override llms.nim_llm.temperature 0.7</code>	Override config at runtime

Agent Lab

Benchmark, diagnose, and iterate on real agent architectures



Lab: Repos, Guides, and Links



Lab GitHub Repo

[Open link ↗](#)

Pre-Lab Setup

[Open link ↗](#)

Lab Guide

[Open link ↗](#)

Lab Roadmap

Eval-driven development: define what good means, measure every change, ship when the bar is met.

Explore

- 1 See the agent loop in action 5 min
- 2 Experience compound probability 10 min

Evaluate

- 3 Compare three architectures 10 min
- 4 Diagnose failures in Phoenix 15 min

Iterate

- 5 Improve an agent with one change 15 min
- 6 Benchmark and compete ongoing

Traces are your data

Every span tree is a training signal. Production debugging starts here.

Define evals first

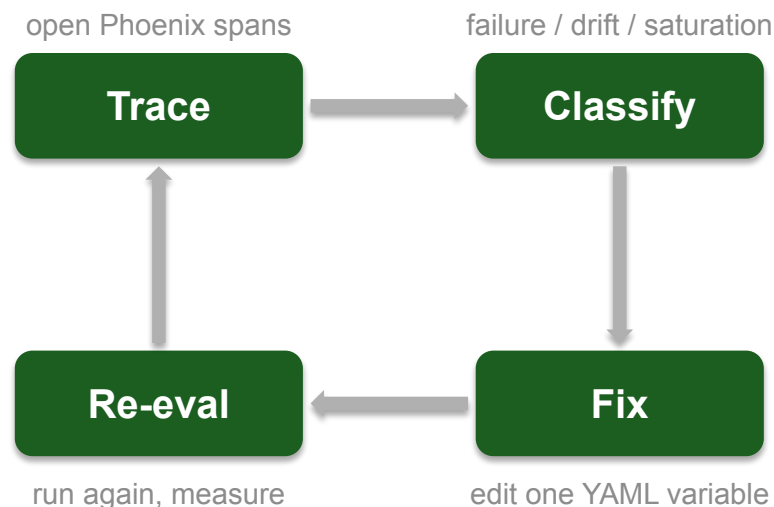
Write the passing criteria before the prompt, like writing tests before code in TDD.

One variable per run

Change the prompt, a tool, or the agent type. Never change two things at once.

The eval-driven inner loop

Every improvement starts with a trace, not a guess



github.com/mnajafian-nv/nat-agent-lab

EDD = Eval-Driven Development | YAML = YAML Ain't Markup Language | Phoenix = Arize Phoenix tracing UI

GAIA: Your Evaluation Benchmark

Real-world questions that require multi-step reasoning and tool use.

Why GAIA is hard

- 1-15 tool calls per question
- Exact-match scoring (strict)
- 3 difficulty levels (L1-L3) [30]
- Files, images, audio, video
- Web search + code execution
- No shortcuts, no memorization

Dev set (tune your agent)

- 165 questions with answers [30]
- CLI: `nat run --level dev --start 1 --end 1`
- See expected vs. actual output
- Iterate: edit YAML, re-run
- Check traces in Phoenix
- No leaderboard submission

Test set (score + compete)

- 301 questions, answers hidden [30]
- Run: benchmark
- 20 Level-1 questions scored
- Auto-submits to HuggingFace
- Team appears on leaderboard
- Built-in agents score 85-90% (our dev-set testing)

Resources

[Paper: arxiv.org/abs/2311.12983](https://arxiv.org/abs/2311.12983)

[Student leaderboard \(submit here\)](#)

[Dataset: huggingface.co/datasets/gaia-benchmark/GAIA](https://huggingface.co/datasets/gaia-benchmark/GAIA)

[Official GAIA leaderboard \(300 questions\)](#)

How Nemo Agent Toolkit Builds the Lab Agents from YAML

Four YAML fields shape the behavior. The tool-calling runtime loop stays the same.

Your config: `gaia_agent.yml`

<code>system_prompt</code>	shapes how the LLM reasons
<code>tool_names</code>	which tools the LLM can call
<code>llm_name</code>	which model serves inference
<code>max_iterations</code>	max loops before stopping

What changes between agents:

Single	generic prompt, all 11 tools
Ultrafast	classification prompt (A-D), all 11 tools
Multi	orchestrator prompt, tools = specialist agents

The runtime loop is shared. The behavior changes through config.

```
# YAML _type: tool_calling_agent
# NAT plugin lookup -> this code

# _build_graph()
graph = StateGraph(state_schema)
graph.add_node("agent", agent_node)
graph.add_node("tool", tool_node)
graph.add_edge("tool", "agent")
graph.add_conditional_edges(
    "agent", conditional_edge,
    {TOOL: "tool", END: "__end__"})
graph.set_entry_point("agent")

# conditional_edge(): the only decision
last = state.messages[-1]
if last.tool_calls:
    return TOOL
# no tool_calls = final answer
return END
```

[Source: NeMo-Agent-Toolkit/nvidia_nat_langchain/.../tool_calling_agent/agent.py](#)

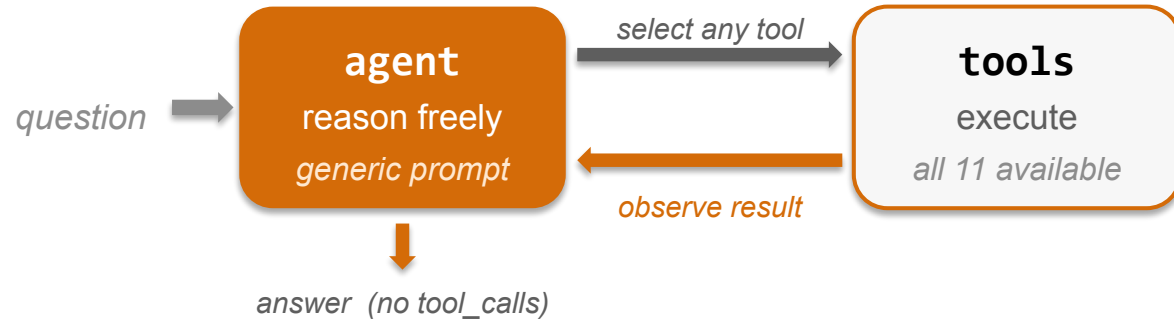
Key insight

The runtime loop stays the same: model call → optional tool call → observation → repeat or final answer

- **Single** vs. **ultrafast**: The runtime loop is shared. The behavior changes through config. The prompt alone drives the performance difference you will measure.
- **Multi** nests the same loop inside specialist agents.
- In the lab, you tune prompt, tools, model, and stopping rules.

Single Agent Architecture

tool_calling_agent: one loop, broad prompt, all 11 tools



Key differences from other agents

- Broad prompt: the model decides which tool to call and when to stop
- All 11 tools available; no routing or classification layer
- 30-iteration cap as a safety net for runaway loops
- MiniMax M2.5 served locally via vLLM on 8xH100
- Baseline config: copy and modify this in Part 5 of the lab

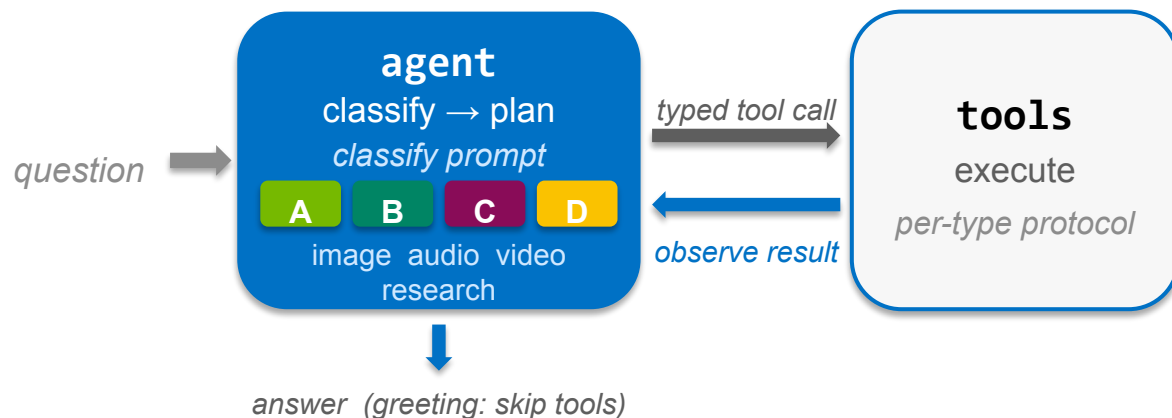
This is the baseline. The next agents change the prompt or composition, not the core tool-calling loop.

```
# single-agent/gaia_agent.yml
workflow:
  _type: tool_calling_agent
  llm_name: nim_llm
  max_iterations: 30
  max_history: 35
  handle_tool_errors: true
  tool_names:
    - internet_search
    - wiki_search
    - current_datetime
    - read_file
    - fetch_url
    - python_executor
    - describe_image
    - describe_image_alt
    - transcribe_audio
    - get_youtube_transcript
    - solve_chess
  system_prompt: >
    You are a precise research
    assistant. You answer
    questions using tools and
    return ONLY the final answer.
```

[Source: nat-agent-lab/single-agent/gaia_agent.yml](https://github.com/nat-agent-lab/single-agent/gaia_agent.yml)

Ultrafast Agent Architecture

Same graph as single. A structured prompt changes the behavior.



Key differences from single

- Same graph, tools, and model as the single-agent baseline
- Prompt asks the model to classify the request into A/B/C/D first
- Each class maps to a recommended tool-use protocol
- Greetings can bypass tools when no tool call is needed
- NAT does not see A/B/C/D; it only sees model outputs and tool calls

A structured prompt can change agent behavior without changing the graph.

```
# ultrafast-agent/gaia_agent_ultrafast.yml
# (same _type, tools, model as single)
system_prompt: >
  You are a precise research
  assistant...

MANDATORY FIRST STEP:
CLASSIFY THE QUESTION.

TYPE A - IMAGE/FILE:
  .png .jpg .pdf .csv ...
TYPE B - AUDIO:
  .mp3 .wav .m4a ...
TYPE C - VIDEO:
  youtube.com/watch ...
TYPE D - RESEARCH:
  Everything else.

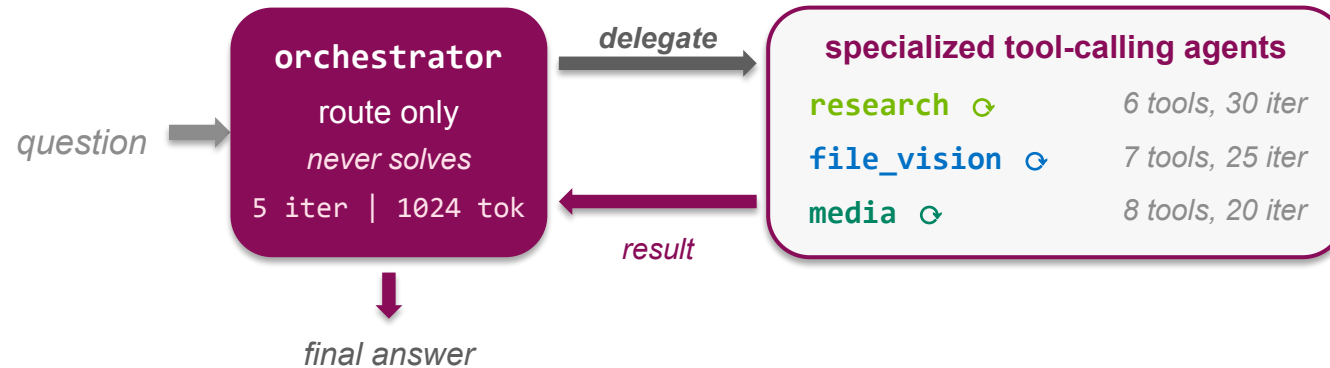
Then follow ONLY the protocol
for that type below.

TYPE A PROTOCOL:
  describe_image / read_file
# B, C, D protocols follow ...
```

Source: [nat-agent-lab/ultrafast-agent/gaia_agent_ultrafast.yml](https://github.com/nat-agent-lab/ultrafast-agent/gaia_agent_ultrafast.yml)

Multi-Agent Architecture

Orchestrator + 3 specialist agents (nested execution)



Key differences from single and ultrafast

- Orchestrator tools are specialist agent workflows, not simple functions
- Each specialist has its own prompt, tools, and iteration budget
- Orchestrator stays lightweight: routes and synthesizes
- Specialists do the deeper research, file, vision, or media work
- Phoenix traces show nested spans: orchestrator → specialist

```
# multi-agent/gaia_agent_multi.yml
workflow:
```

```
  _type: tool_calling_agent
  llm_name: orchestrator_llm
  max_iterations: 5
  tool_names:
    - research_agent
    - file_vision_agent
    - media_agent
  system_prompt: >
    You are a ROUTING-ONLY
    orchestrator. You NEVER
    answer questions yourself.
```

```
functions:
```

```
  research_agent:
    _type: tool_calling_agent
    llm_name: specialist_llm
    max_iterations: 30
    tool_names:
      - internet_search
      - wiki_search
      - fetch_url
      # + 3 more tools
  # file_vision + media defined similarly
```

Multi-agent buys specialization, but adds routing overhead and another failure point.

[Source: nat-agent-lab/multi-agent/gaia_agent_multi.yml](https://github.com/nat-agent-lab/multi-agent/gaia_agent_multi.yml)

Anatomy of an Agent Trace

Each run produces a trace. Tool calls, model calls, and workflow steps appear as spans.

What you see when you click a trace

```
<workflow>                                (root: NAT orchestration)
└─ <workflow>                             (LangChain StateGraph)
    └─ internet_search                    (agent searched the web)
    └─ fetch_url                         (fetched a specific page)
    └─ python_executor                   (ran a calculation)
    └─ python_executor                   (produced FINAL ANSWER)
```

Level 1 question (simpler): ~5 spans, ~25 s

Level 2 question (harder): 10-15 spans, 1-3 min

Click any span to see three tabs:

Input

Prompt, tool arguments, query, or code

Output

Model text, tool result, code output, or error

Attributes

Tokens, latency, model, status, metadata

First look: three-number snapshot

- 1 Total traces** how many agent runs you have
- 2 P50 latency** median (half finish faster)
- 3 P99 latency** worst case (1 in 100 slower)

Large P99/P50 gaps point to tail-latency problems.

Where to look in Phoenix

Traces	One row per question. Start here.
Spans > All	All spans flattened. Filter by name.
Filter: name	name == 'internet_search'
Filter: error	status_code == 'ERROR'
Attributes	input_tokens, output_tokens
Output	<workflow> Output has <think> block

Reading Failure Signals in Traces

Most failures fall into three patterns. Classify the trace before changing the config.

[Failure]

Wrong on first attempt

What you see in the trace:

- Wrong tool selected or no tool call
- Vague query ('building' vs 'tallest building SF')
- Answers from memory, skips search
- Extra text around FINAL ANSWER

Config fix:

- Add tool-selection guidance to prompt
- Add 'never answer from memory' rule
- Add FINAL ANSWER formatting rule

[Drift]

Correct start, degrades over steps

What you see in the trace:

- Tool echo: same search query repeated
- Code spiral: 7+ python_executor calls
- Token growth across turns
- Repeated verification without new evidence

Config fix:

- Add stopping rule or duplicate-search rule
- Tune repetition controls if supported
- Prompt: 'do not search same fact twice'
- Reduce max_results on search tools

[Saturation]

System crash or timeout

What you see in the trace:

- Many repeated tool spans
- Trace latency much higher than typical runs
- No FINAL ANSWER produced
- Error status (red X) on late spans

Config fix:

- Tune max_iterations based on trace evidence
- Remove unused tools (shorter prompt)
- Temporarily fail fast during debugging

trace → classify → fix → re-eval (the eval-driven inner loop). One variable at a time.



Thank You

Maryam Najafian, PhD
Brendan Murphy



References

- [1] Yao et al., "ReAct: Synergizing Reasoning and Acting in Language Models," ICLR 2023.
- [2] Xu et al., "ReWOO: Decoupling Reasoning from Observations for Efficient Augmented LMs," 2023.
- [3] Shinn et al., "Reflexion: Language Agents with Verbal Reinforcement Learning," NeurIPS 2023.
- [4] Dhuliawala et al., "Chain-of-Verification Reduces Hallucination in LLMs," Meta AI, 2023.
- [5] Wang et al., "CodeAct: Executable Code Actions Elicit Better LLM Agents," ICML 2024.
- [6] Yang et al., "SWE-Agent: Agent-Computer Interfaces Enable Automated SW Engineering," NeurIPS 2024.
- [7] Xie et al., "OSWorld: Benchmarking Multimodal Agents for Open-Ended Tasks," NeurIPS 2024.
- [8] Hong et al., "MetaGPT: Meta Programming for Multi-Agent Collaboration," ICLR 2024.
- [9] Li et al., "CAMEL: Communicative Agents for Mind Exploration of Large LMs," NeurIPS 2023.
- [10] Schluntz & Barry (Anthropic), "Building Effective Agents," blog, Dec 2024.
- [11] Anthropic, "Developing a Computer Use Model," research report, Oct 2024.
- [12] Schulman et al., "Proximal Policy Optimization Algorithms," arXiv, 2017.
- [13] Rafailov et al., "DPO: Direct Preference Optimization," NeurIPS 2023.
- [14] Shao et al., "DeepSeekMath: Pushing the Limits of Math Reasoning" (introduces GRPO), arXiv, 2024.
- [15] Zheng et al., "GSPO: Group Sequence Policy Optimization," arXiv, 2025.
- [16] Gartner Research, "Enterprise AI Agent Forecast," market report, 2025.
- [17] OWASP Foundation, "Top 10 for Agentic Applications," standard, 2025.
- [18] Dao et al., "Agentic Design Patterns: A System-Theoretic Framework," NeurIPS 2025.
- [19] Xi et al., "Adaptation of Agentic AI: Post-Training, Memory, and Skills," arXiv, 2025.
- [20] Li & Wang, "Memory for Autonomous LLM Agents," arXiv, Mar 2026.
- [21] Weidener et al., "OpenClaw, Moltbook, and ClawdLab," arXiv, Feb 2026.
- [22] Abou Ali & Dornaika, "Agentic AI: Architectures, Applications, and Future Directions," arXiv, 2025.
- [23] NVIDIA, "NeMo Agent Toolkit Documentation," docs.nvidia.com/nemo/agent-toolkit, 2026.
- [24] Packer et al., "MemGPT: Towards LLMs as Operating Systems," NeurIPS 2024.
- [25] Anthropic Engineering, "Model Context Protocol (MCP)," open spec, Nov 2024.
- [26] Surapaneni et al. (Google), "Agent-to-Agent Protocol (A2A)," open spec, Apr 2025.
- [27] Fournay et al., "Magentic-One: A Generalist Multi-Agent System," Microsoft, Nov 2024.
- [28] Wu et al., "AutoGen: Multi-Agent Conversation," COLM 2024.

References (cont.)

- [29] Guo et al., "DeepSeek-R1: Incentivizing Reasoning in LLMs via RL," Nature, 2025.
- [30] Maia et al., "GAIA: A Benchmark for General AI Assistants," ICLR 2024.
- [31] OpenAI Safety, "Operator System Card" (Computer-Using Agent), Jan 2025.
- [32] Qian et al., "ToolRL: Reward is All Tool Learning Needs," NeurIPS 2025.
- [33] Zhang et al., "G-Memory: Hierarchical Memory for Multi-Agent Systems," arXiv, 2025.
- [34] Xu et al., "A-MEM: Agentic Memory for LLM Agents" (Zettelkasten-style linking), arXiv, 2025.
- [35] Zhang et al., "Agentic Plan Caching: Test-Time Memory for Cost-Efficient Agents," NeurIPS 2025.
- [36] Arnav et al., "CoT Red-Handed: Stress Testing Chain-of-Thought Monitoring," NeurIPS 2025.
- [37] Lin et al., "SE-Agent: Self-Evolution Trajectory Optimization," arXiv, 2025.
- [38] Yang et al., "KVLink: Efficient KV Cache Reuse," arXiv, 2025.
- [39] Oliaro et al., "SuffixDecoding: Speculative Decoding for Agentic Workloads," NeurIPS 2025.
- [40] Luo et al., "AgentAuditor: Human-Level Safety and Security Evaluation for LLM Agents," arXiv, 2025.
- [41] Sun et al., "HyGen: Elastic Online-Offline Request Co-location," NeurIPS 2025.
- [42] Swanson et al., "The Virtual Lab of AI Agents Designs New SARS-CoV-2 Nanobodies," Nature, 2025.
- [43] Savchenko et al., "LLM-Driven Multi-Agent Pipeline for Drug Discovery," AI4Research @ AAAI, 2025.
- [44] Linardon et al., "Citation Hallucinations in GPT-4o," JMIR Mental Health, 2025.
- [45] NVIDIA, "Agent-Aware Inference with NeMo Agent Toolkit + Dynamo," GTC 2026.
- [46] OpenAI, "A Practical Guide to Building Agents," business guide, 2025.
- [47] Bhatia et al. (AWS), "Strands Agents and the Model-Driven Approach," AWS Open Source Blog, 2025.
- [48] Wang et al., "OpenHands LM 32B: A Strong, Open Coding Agent Model," openhands.dev, Nov 2025.
- [49] Google Cloud, "Google Cloud Study Reveals 52% of Executives Say Their Organizations Have Deployed AI Agents," press release, Sept. 2025.
- [50] McKinsey, "The State of AI in 2025: Agents, Innovation, and Transformation," Nov. 2025.
- [51] Cognition, "Devin's 2025 Performance Review: Learnings From 18 Months of Agents At Work," Nov. 2025.
- [52] Mastercard, "Santander and Mastercard complete Europe's first live end-to-end payment executed by an AI agent," Mar. 2026.
- [53] Epic, "Real Results, Right Now: How Epic AI Is Reducing Costs, Improving Care, and Helping Patients," Mar. 2026.
- [54] OpenAI, "Introducing GPT-5.4," Mar. 2026.
- [55] Horváth, "Interview with Christiane Vorspel, Commerzbank," 2026.
- [56] PR Newswire, "Lightwheel and PeritasAI Announce Strategic Partnership to Bring Physical AI into Perioperative Workflows," Apr. 2026.
- [57] CIO, "88% of AI pilots fail to reach production, but that's not all on IT," Mar. 2025.
- [58] Rasa, "Why AI Agent Demos Take 4 Months to Reach Production," 2026.
- [59] Cemri et al., "Why Do Multi-Agent LLM Systems Fail?", NeurIPS 2025 / arXiv 2503.13657.